BLOCK CHAIN BASED ELECTRONIC VOTING SYSTEM

A PROJECT REPORT

Submitted to the APJ Abdul Kalam Technological University

in partial fulfillment of requirements for the award of degree

Bachelor of Technology

in

Computer Science and Engineering

by

ADEEBA ABDUL VAHID	**KNP23CS011**
AVANI P	**LKNP23CS114**
THARA A	**LKNP23CS117**

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

COLLEGE OF ENGINEERING KARUNAGAPPALLY

KOLLAM, KERALA

October 2026

COLLEGE OF ENGINEERING KARUNAGAPPALLY

Thodiyoor – 690523

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Vision of Department

To be renowned centre for education, research, and innovation in the frontier areas of Computer Science and Engineering.

Mission of Department

- Establish and maintain an operational environment to acquire, impart, create and apply knowledge in Computer Science and Engineering and inter-disciplinary.
- Serve as a resource centre for innovation in design and development of hardware and software.
- Inculcate leadership qualities, professional ethics and a sense of social commitment.

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
COLLEGE OF ENGINEERING KARUNAGAPPALLY
2026–2027

CERTIFICATE

This is to certify that the report entitled **BLOCK CHAIN BASED ELECTRONIC VOTING SYSTEM** submitted by **ADEEBA ABDUL VAHID** (KNP23CS011), **AVANI P** (LKNP23CS114), and **THARA A** (LKNP23CS117), to the Department of Computer Science and Engineering in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in **Computer Science and Engineering** of the APJ Abdul Kalam Technological University is a bonafide record of the project work carried out by them under our guidance and supervision. This report in any form has not been submitted to any other University or Institute for any purpose.

Mr. Anwar Sadath
(Project Guide)
Assistant Professor
Dept. of CSE
CE, Karunagappally

Dr. Smitha P
(Project Coordinator)
Associate Professor and HOD
Dept. of CSE
CE, Karunagappally

DECLARATION

We, **ADEEBA ABDUL VAHID** (KNP23CS011), **AVANI P** (LKNP23CS114), and **THARA A** (LKNP23CS117), hereby declare that the project report entitled **BLOCK CHAIN BASED ELECTRONIC VOTING SYSTEM**, submitted for partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering of the APJ Abdul Kalam Technological University, Kerala, is a bonafide work done by us under the supervision of **Mr. Anwar Sadath** (Assistant Professor, Dept. of CSE, College of Engineering Karunagappally).

This submission represents our ideas in our own words, and where ideas or words of others have been included, we have adequately and accurately cited and referenced the original sources.

We also declare that we have adhered to the ethics of academic honesty and integrity and have not misrepresented or fabricated any data, idea, fact, or source in our submission. We understand that any violation of the above will be a cause for disciplinary action by the Institute and/or the University and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been obtained. This report has not previously formed the basis for the award of any degree, diploma, or similar title of any other University or Institute.

ADEEBA ABDUL VAHID (KNP23CS011)

AVANI P (LKNP23CS114)

THARA A (LKNP23CS117)

Karunagappally

28-10-2026

Abstract

Conventional paper-based and centralized electronic voting systems suffer from systemic vulnerabilities, including ballot tampering, administrative opacity, slow vote tabulation, and double voting. Centralized electronic databases introduce a single point of failure and require electorates to place blind trust in a single administrative authority. To overcome these limitations, this project presents the design and implementation of a **Block Chain Based Electronic Voting System**, realized as a decentralized application (Voting DApp) built upon the Ethereum blockchain network.

The core voting logic is governed by an immutable **Solidity** smart contract that enforces strict role-based access control, administrator-managed voter whitelisting, dynamic candidate management, and smart-contract-level mitigation of duplicate voting. An atomic evaluation mechanism ensures that a ballot is accepted if and only if the voter's Ethereum address is verified as registered and has not previously cast a vote. Casting a vote atomically marks the voter's participation flag and increments the candidate tally within a single transaction, eliminating race conditions and direct contract bypassing. The client interface is engineered using **React.js** and **Web3.js**, seamlessly interfacing with the **MetaMask** browser extension for asymmetric cryptographic authentication and non-repudiation.

To address real-world deployment constraints, the system evaluates consensus models, identifying a consortium **Proof-of-Authority (PoA)** architecture as an optimal mechanism to eliminate unpredictable public gas fees and provide deterministic low-latency transaction processing during bursty polling windows. Furthermore, the report presents a comprehensive academic survey of fifteen recent blockchain voting architectures spanning multimodal biometrics, gesture authentication, blind signatures, and differential privacy, identifying critical scalability, cryptographic anonymity, and voter verification trade-offs. The resulting system establishes an auditable, decentralized, and tamper-resilient electoral platform.

Acknowledgement

We take this opportunity to express our deepest gratitude and sincere thanks to everyone who helped us to complete this project work and report successfully.

We express our sincere thanks to **Dr. Smitha Dharan**, Principal, College of Engineering Karunagappally, Kollam, and **Dr. Smitha P**, Head of Department, Computer Science and Engineering, College of Engineering Karunagappally, Kollam, for providing us with all the necessary infrastructural facilities, high-performance computing resources, and institutional encouragement.

We would like to express our profound sense of gratitude and indebtedness to our project guide **Mr. Anwar Sadath**, Assistant Professor, Computer Science and Engineering, College of Engineering Karunagappally, for his insightful guidance, constant encouragement, rigorous critiques, and invaluable mentorship throughout the entire lifecycle of this project.

We also express our sincere gratitude and indebtedness to our project coordinator **Dr. Smitha P**, Associate Professor and HOD, Computer Science and Engineering, College of Engineering Karunagappally, for her continuous coordination, constructive reviews, and valuable support during each phase of this project.

We also convey our sincere appreciation to all the faculty members and technical staff of the Computer Science and Engineering for their kind cooperation, constructive academic advice, and technical support during our design and development phases.

Finally, we express our wholehearted thanks to our parents, family members, and friends who provided unwavering moral support, patience, and encouragement that contributed immensely to the successful realization of this project.

ADEEBA ABDUL VAHID (KNP23CS011)

AVANI P (LKNP23CS114)

THARA A (LKNP23CS117)

Contents

Abstract i

Acknowledgement ii

List of Figures vi

List of Tables vii

List of Symbols and Abbreviations viii

1 Introduction 1

1.1 Background and Motivation 1

1.2 The Blockchain Trust Paradigm 2

1.3 The Double-Voting Challenge in Electronic Voting 3

1.4 Project Objectives 3

1.5 Scope and Organization of the Report 4

2 Literature Review 6

2.1 Blockchain-Based Voting Frameworks and Consensus Design 6

2.2 Biometric and Gesture-Based Authentication 7

2.3 Privacy and Cryptographic Techniques 8

2.4 Surveys, Trust and Governance Studies 9

2.5 Smart Contract Approaches and Network Support 10

2.6 Synthesis and Research Gap Identification 11

3 Problem Definition 13

3.1 Problem Statement 13

3.2 Threat Model and Attack Vectors 13

3.3 System Boundaries and Operational Assumptions 14
3.4 Mathematical Formulation and Governing Equations 15
3.4.1 Cryptographic Hash Chaining and Block Integrity 15
3.4.2 Atomic Double-Voting Mitigation Predicate 15
3.4.3 Cumulative Vote Tally and Upper Bound Invariant 16

4 Requirements Specification **17**
4.1 Functional Requirements 17
4.1.1 Administrator Module 17
4.1.2 Voter Module 18
4.1.3 Smart Contract and Ledger Module 18
4.2 Non-Functional Requirements 19
4.3 Hardware and Software Specifications 19

5 System Design **21**
5.1 Multi-Tier System Architecture 21
5.2 Proposed Operational Methodology Pipeline 22
5.3 Voter Registration and Whitelisting Workflow 23
5.4 End-to-End Ballot Sequence Flow 24
5.5 Complete System Operational Flowchart 25
5.6 Algorithm 26
5.7 Block Structure and Cryptographic Chaining 31
5.8 Smart Contract State Schemas and Structs 32
5.9 Level-1 Data Flow Diagram (DFD) 33

6 Design and Methodology **35**
6.1 Solidity Smart Contract Architecture 35
6.2 Staged Execution and Double-Voting Mitigation Pipeline 36
6.3 Front-End DApp Architecture with React.js and Web3.js 37
6.4 MetaMask Cryptographic Handshake and Non-Repudiation 38
6.5 Consensus Mechanism Analysis 38

7 Preliminary Analysis **40**
7.1 Comparative Platform Evaluation 40
7.2 Consensus Models Comparison 40

7.3 Gas Consumption and Cost Analysis 41
7.4 Security Vulnerability Analysis 41

8 Feasibility Study **43**
8.1 Technical Feasibility 43
8.2 Operational Feasibility 43
8.3 Economic Feasibility 44
8.4 Project Implementation Timeline 44

9 Conclusion and Future Scope **46**
9.1 Summary of Achievements 46
9.2 System Limitations 47
9.3 Future Scope and Research Trajectories 48

References **49**

List of Figures

5.1 Multi-tier system architecture of the blockchain-based electronic voting decentralized application (Voting DApp). 22

5.2 Proposed operational methodology pipeline detailing the five stages of the election lifecycle. 23

5.3 Detailed layered workflow for voter whitelisting, MetaMask connection, and eligibility verification. 24

5.4 End-to-end system sequence flow from ballot selection to block sealing and real-time Web3 tally notification. 24

5.5 Comprehensive system flowchart detailing administrator actions, voter verification branches, and atomic double-voting mitigation. 25

5.6 Cryptographic block structure and hash chaining mechanism illustrating ledger immutability. 31

5.7 Smart contract state schemas, data structs, and relational mappings governing voters and candidates. 32

5.8 Level-1 Data Flow Diagram (DFD) for the blockchain-based electronic voting system. 33

6.1 Staged execution flow and atomic double-voting mitigation pipeline within the EVM runtime. 36

8.1 Gantt chart illustrating the project development timeline and milestone schedule (Jul 2026 – Mar 2027). 45

List of Tables

2.1 Systematic Comparative Analysis of Surveyed Blockchain Electronic Voting Literature 11

4.1 System Hardware and Software Technical Specifications 20

5.1 Data Flow Dictionary detailing communication flows and integrity constraints. 34

7.1 Comparative Analysis of Leading Blockchain Platforms for Electronic Voting 40

7.2 Consensus Mechanism Evaluation Across Performance and Security Metrics 41

7.3 Empirical Smart Contract Gas Consumption Analysis 41

List of Symbols and Abbreviations

Abbreviations

DApp	Decentralized Application
EVM	Ethereum Virtual Machine
PoA	Proof of Authority
PoS	Proof of Stake
PoW	Proof of Work
DPoS	Delegated Proof of Stake
PBFT	Practical Byzantine Fault Tolerance
ABI	Application Binary Interface
RPC	Remote Procedure Call
JSON-RPC	JavaScript Object Notation Remote Procedure Call
IPFS	InterPlanetary File System
SSI	Self-Sovereign Identity
SHA-256	Secure Hash Algorithm 256-bit
Keccak-256	Cryptographic Hash Standard utilized in Ethereum EVM
P2P	Peer-to-Peer Network
TPS	Transactions Per Second
ECDSA	Elliptic Curve Digital Signature Algorithm
RFID	Radio Frequency Identification
CNN	Convolutional Neural Network
SRS	System Requirements Specification
DFD	Data Flow Diagram
UI/UX	User Interface / User Experience

Mathematical Notations

B_i	Sequential block entity positioned at height index i in the chain
$H(B_i)$	Cryptographic block digest hash computed for block B_i
T_i	Unix epoch timestamp corresponding to block creation time
Tx_i	Ordered set of validated transaction payloads encapsulated in block B_i
n_i	Cryptographic nonce value resolving mining difficulty or consensus sealing
$\|\|$	Byte-level string concatenation operator
$\mathcal{V}$	Universal set of eligible voter Ethereum addresses ($\mathcal{V} \subset \mathbb{Z}_{2^{160}}$)
C	Indexed set of certified election candidates ($C = \{1, 2, \ldots, K\}$)
v	Individual voter Ethereum wallet address ($v \in \mathcal{V}$)
c	Specific candidate identifier selected on the ballot ($c \in C$)
registered(v)	Smart contract state mapping certifying voter registration status
voted(v)	Smart contract state mapping indicating prior ballot submission
Accept(v)	Atomic binary verification predicate governing ballot acceptance
$N(c)$	Cumulative tallied vote total recorded for candidate c
vote(v)	Candidate identifier selected and submitted by voter v
$K_{\text{priv}}, K_{\text{pub}}$	Asymmetric elliptic curve (secp256k1) private signing key and public key
$\text{Sig}_{K_{\text{priv}}}(m)$	Cryptographic digital signature over transaction payload m
G_{limit}	Upper bound on computational gas allocated for contract execution
G_{used}	Exact volume of gas consumed by EVM opcode execution
G_{price}	Gas price valuation specified per unit of computation (Gwei)

Chapter 1

Introduction

Democratic governance is fundamentally anchored upon the integrity, transparency, and public verifiability of electoral processes. The legitimacy of any elected administration depends upon ensuring that every eligible citizen is guaranteed exactly one non-repudiable vote, that ballots cannot be forged, manipulated, or discarded, and that final tallies are immune to post-election tampering. Despite centuries of institutional evolution, conducting secure, accessible, and transparent elections remains one of the most contentious socio-technical challenges confronting modern societies.

1.1 Background and Motivation

Traditional paper-based voting methodologies, while conceptually simple, suffer from severe systemic vulnerabilities. Paper ballot administration requires massive logistical mobilization, physical transportation of paper ballot boxes, and centralized manual tallying procedures that are vulnerable to booth capturing, ballot stuffing, physical loss, counting discrepancies, and deliberate administrative tampering. Furthermore, the protracted time required to manually count and audit millions of physical paper ballots introduces periods of severe uncertainty, dispute, and civil unrest.

To mitigate logistical latency and human counting fatigue, many jurisdictions adopted first-generation electronic voting machines (EVMs) and centralized online web-based voting platforms. While electronic voting drastically reduces ballot printing costs and accelerates result compilation, conventional e-voting systems predominantly store voting records within centralized relational databases managed by a single election authority. This centralized architecture introduces a catastrophic single point

of failure:

(i) **Single Point of Centralized Control:** Database administrators, system engineers, or malicious actors with privileged root access can manipulate ballot tables, inject fictitious votes, or alter candidate tallies without leaving an externally verifiable, cryptographically provable audit trail.

(ii) **Opaque Blind-Trust Model:** Electorates are required to place blind, unverifiable faith in the honesty of the centralized election commission and proprietary machine software.

(iii) **Double-Voting Vulnerabilities:** In distributed web-based voting portals, network race conditions, duplicate session logins, and concurrency bugs frequently allow malicious actors to cast multiple ballots under a single identity before database row locks resolve.

(iv) **Vulnerability to Denial of Service and Data Loss:** Centralized server infrastructures remain vulnerable to distributed denial-of-service (DDoS) assaults, server downtime, and catastrophic hardware failures during bursty polling periods.

1.2 The Blockchain Trust Paradigm

Blockchain technology introduces a decentralized trust model that eliminates reliance on trusted intermediaries. A blockchain operates as a peer-to-peer, append-only distributed ledger wherein transactional records are grouped into chronologically sequenced blocks linked cryptographically through collision-resistant hash functions (such as SHA-256 and Keccak-256). Once a block is committed to the distributed chain through network consensus, altering historical ballot data would require computationally recalculating all subsequent block hashes across a majority of distributed nodes—an operation that is computationally infeasible and immediately detectable by all participating peers.

By integrating **Smart Contracts**—self-executing, immutable program routines running within the Ethereum Virtual Machine (EVM)—the procedural rules, voter eligibility checks, candidate registries, and counting algorithms can be automated

without subjective human intervention. Smart contracts eliminate opaque counting rooms; once deployed, their bytecoded logic is executed identically across every node in the decentralized network.

1.3 The Double-Voting Challenge in Electronic Voting

Double voting represents one of the most fatal threats to election legitimacy. Double voting occurs when a registered voter succeeds in casting more than one ballot, either intentionally through malicious browser automation and concurrent transaction submissions, or through system synchronization lags across distributed polling nodes. In centralized client-server voting architectures, anti-duplicate validation is typically enforced at the presentation layer (e.g., disabling a UI button after clicking) or via application middleware. Such mechanisms fail when malicious actors interact directly with server APIs or exploit race conditions in concurrent database writes.

The proposed system addresses double voting at the immutable smart contract layer. By coupling an atomic verification predicate within the Solidity smart contract with asymmetric wallet-based authentication via MetaMask, duplicate ballot submissions are rejected directly by the EVM state transition engine before any state mutation or tally aggregation can occur.

1.4 Project Objectives

The primary objective of this project is to design, implement, and analyze an end-to-end, secure, and transparent electronic voting system leveraging Ethereum smart contracts, Web3.js middleware, and a React.js client interface. The explicit engineering and research objectives are as follows:

- **Smart-Contract-Level Double-Voting Mitigation:** Develop an atomic state evaluation mechanism within Solidity that enforces strict single-vote casting (Accept(v) logic) and makes duplicate voting computationally impossible even under direct RPC invocation.

- **Role-Based Administrative Access Control:** Implement strict cryptographic authorization boundaries separating the election administrator (responsible for

candidate nomination and voter whitelisting) from registered voters (authorized solely to cast a single ballot).

- **Asymmetric Wallet Authentication:** Integrate MetaMask and Web3.js to enable voters to securely authenticate and cryptographically sign ballot transactions using secp256k1 elliptic curve digital signatures without exposing private keys.
- **Real-Time Public Auditability and Result Transparency:** Eliminate post-polling counting delays by providing an automated tallying engine that reflects certified results on an auditable ledger in real time through event subscriptions.
- **Consensus and Scalability Optimization:** Formally evaluate consensus mechanisms, establishing the operational viability of Proof-of-Authority (PoA) consortium networks to eliminate gas fee volatility and ensure predictable, low-latency transaction throughput.
- **Comprehensive Academic Survey:** Synthesize a systematic comparative review of fifteen recent state-of-the-art blockchain voting architectures spanning multimodal biometrics, gesture recognition, blind signatures, and differential privacy.

1.5 Scope and Organization of the Report

This project report encompasses the complete architectural design, mathematical modeling, smart contract engineering, front-end development, security vulnerability auditing, and consensus analysis of the proposed Voting DApp. The remainder of this report is organized as follows:

- **Chapter 2 (Literature Review):** Synthesizes fifteen foundational and recent peer-reviewed investigations on blockchain e-voting frameworks, biometrics, cryptography, and governance, concluding with an extensive comparative taxonomy and identified research gaps.
- **Chapter 3 (Problem Definition):** Details the formal problem statement, threat vectors, system boundaries, and governing mathematical equations for cryptographic block chaining, atomic ballot acceptance, and vote tallying.

- **Chapter 4 (Requirements Specification):** Details the functional and non-functional requirements governing administrators, voters, smart contracts, and network infrastructure, alongside hardware and software specifications.

- **Chapter 5 (System Design):** Elaborates the multi-tier system architecture, operational pipelines, voter registration workflows, end-to-end sequence flows, comprehensive operational flowchart, the formal 12-stage operational algorithm, block structures, relational state schemas, and Level-1 Data Flow Diagrams (DFD).

- **Chapter 6 (Design and Methodology):** Analyzes Solidity smart contract implementation, modifiers, staged transaction verification, React/Web3.js front-end integration, MetaMask ECDSA signing, and consensus model trade-offs.

- **Chapter 7 (Preliminary Analysis):** Conducts comparative evaluations across blockchain platforms (Ethereum, Hyperledger, Polygon), consensus mechanisms (PoW, PoS, PoA), EVM gas expenditures, and smart contract vulnerability mitigation.

- **Chapter 8 (Feasibility Study):** Evaluates technical, operational, and economic feasibility dimensions, accompanied by the complete project milestone Gantt chart.

- **Chapter 9 (Conclusion and Future Scope):** Summarizes major technical milestones, discusses remaining real-world constraints (e.g., key custody, ballot privacy), and charts prospective research trajectories including Zero-Knowledge Proofs and Layer-2 scaling.

Chapter 2

Literature Review

The application of blockchain technology to electronic voting has garnered extensive academic attention over the past decade. Researchers have investigated blockchain architectures to address the vulnerabilities of centralized systems while striving to satisfy classical electoral criteria: eligibility, uniqueness, ballot secrecy, verifiability, and coercion resistance. This chapter synthesizes recent advancements across five thematic dimensions: blockchain voting frameworks, biometric and gesture authentication, cryptographic privacy techniques, surveys and governance studies, and smart contract design paradigms.

2.1 Blockchain-Based Voting Frameworks and Consensus Design

Early investigations focused on adapting core blockchain ledgers to record votes as immutable transactions. Farooq et al. [2] proposed a comprehensive Voting Management System utilizing a custom blockchain with a pluggable consensus layer. In their framework, a web and mobile DApp verifies a voter's national identification credentials, verifies that the voter has not previously participated, converts the cast ballot into an encrypted transaction, and commits it to the chain. A proprietary Chain Security Algorithm continuously re-audits chain integrity upon block inclusion. Although their simulation demonstrated manageable response latencies and linear chain growth up to six million transactions, the architecture requires continuous internet connectivity, lacks a biometric verification layer, and mitigates 51% attacks

only through centralized monitoring of authorized mining nodes.

Shahzad and Crowcroft [3] observed that polling periods are intrinsically bursty, characterized by high transaction volumes concentrated within a fixed window of 8–12 hours. Consequently, resource-heavy Proof-of-Work (PoW) consensus is computationally wasteful and impractical. They developed an adjusted consortium blockchain wherein certified presiding officers with verified cryptographic identities generate blocks during polling hours, sealing them at the close of voting using a proof-of-completeness hashing mechanism. While tamper-evident and computationally lightweight, the framework remains purely conceptual, lacks an empirical prototype, and retains significant administrative centralization within the election commission.

Kshetri and Voas [4] conceptualized a wallet-and-coin tokenized voting model. In this formulation, each eligible voter receives exactly one non-fungible or restricted digital token representing a single vote; casting a ballot corresponds to transferring this coin into a certified candidate's wallet address. Their analytical study evaluated practical impediments to real-world deployment, concluding that software defect rates in smart contracts, voter digital literacy barriers, high broadband infrastructure requirements, and political resistance make public blockchain voting unprepared for national elections without hybrid physical safeguards.

Seftyanto et al. [5] architected a permissioned election infrastructure tailored for the Indonesian electoral context utilizing Hyperledger Fabric. In their multi-organizational topology, each geographic province functions as an autonomous peer organization. Voters authenticate at polling booths using an electronic national identity card (e-ID) and a biometric fingerprint scanner, with cast votes encapsulated as immutable `Ballot` assets within chaincode. However, their contribution remains a preliminary design without throughput benchmarks, latency evaluations, or stress testing, and relies heavily on costly dedicated hardware at every polling venue.

2.2 Biometric and Gesture-Based Authentication

Voter authentication represents the most vulnerable attack surface in decentralized voting. A system that relies solely on private key possession cannot verify whether the authorized citizen or an unauthorized third party is operating the wallet. To bridge this gap, Ajao et al. [6] integrated a multimodal biometric hardware apparatus

with an Ethereum Proof-of-Stake (PoS) network. Their setup deployed a custom Raspberry Pi device equipped with facial recognition, optical fingerprint scanning, and high-frequency RFID credential readers. Biometric embeddings and ballot selections were encrypted using Paillier homomorphic encryption to ensure end-to-end data confidentiality. In a live university election involving 2,423 students, the system achieved a remarkable 1,424.5 transactions per second (TPS), an average mining latency of 7.02 seconds, and zero false acceptance (FAR) and false rejection rates (FRR). Nevertheless, the operational cost of manufacturing and distributing specialized biometric scanning hardware to distributed polling stations, coupled with Ethereum gas fees per transaction, poses substantial barriers for wide-scale public adoption.

Mahmood et al. [7] focused on accessibility and assistive voting technologies for motor-impaired and elderly electorates. Their system tracks real-time facial landmarks and hand gestures via consumer webcams, allowing voters to navigate virtual ballot keyboards through subtle facial movements (e.g., eyebrow raises, nose gestures, or pinch gestures). Integrated with a PoW blockchain backend, the system demonstrated 100% precision and 99% recall across controlled trials. However, system accuracy degraded significantly under suboptimal lighting conditions, finger gestures exhibited lower tracking fidelity than facial landmarks, and elderly participants exhibited longer voting completion times.

2.3 Privacy and Cryptographic Techniques

Balancing the radical transparency of public distributed ledgers with the strict constitutional mandate of voter ballot secrecy remains a central cryptographic paradox. Pérez García et al. [8] addressed this challenge by implementing an ElGamal-based blind signature protocol over Hyperledger Fabric with Practical Byzantine Fault Tolerance (PBFT) consensus. In their protocol, a central electoral authority cryptographically signs an blinded, encrypted ballot without deciphering its content, completely uncoupling the voter's identity from the cast choice. Ballots are divided into two cryptographic shares such that neither share alone reveals the voter's candidate selection. While simulations confirmed total unlinkability and minimal computational overhead per client, transaction processing latency increased under high concurrency, and the centralized authority retains significant trust for blinded token issuance.

Baniata and Caluna [9] proposed *BP-Vot*, a privacy-preserving framework deployed on Hyperledger Besu. *BP-Vot* combines Self-Sovereign Identity (SSI) verifiable credentials with a (k, ε)-differential privacy mechanism to inject controlled mathematical noise into aggregate vote counts, guaranteeing plausible deniability for individual ballot selections. Their prototype achieved approximately one second transaction latency and demonstrated over 98% tally approximation accuracy. However, evaluation was restricted to a three-node cluster processing up to 50,000 votes, fundamentally relies upon the honesty of the credential-issuing authority, and depends on global standardization of digital identity wallets.

2.4 Surveys, Trust and Governance Studies

Benabdallah et al. [10] conducted a systematic literature review analyzing blockchain voting proposals published between 2010 and 2021 across five election lifecycle phases: initialization, voter identification, vote casting, tallying, and result verification. Their study concluded that blockchain architectures cannot completely eliminate centralized authorities responsible for voter eligibility rosters, that public proof-of-work blockchains fail to achieve the throughput required for national elections, and that receipt-freeness and coercion resistance remain largely unresolved across existing prototypes.

Quzmar et al. [11] performed a comparative survey of eight prominent blockchain e-voting architectures, finding that the vast majority remain academic prototypes tested exclusively in local simulation environments. They identified key management vulnerabilities, lack of standard blockchain protocols, and smart contract security bugs as primary impediments.

Sah et al. [12] conducted a comprehensive architectural review of blockchain voting implementations, focusing on developer toolchains including Truffle, Ganache, and MetaMask. Synthesizing pilot election data, they reported an 87% reduction in reported electoral fraud incidents and a 10% increase in voter participation, while emphasizing that transaction fees and rural internet connectivity bottlenecks persist.

Barelli et al. [13] catalogued attacks on real-world e-voting deployments and defined six fundamental security properties: integrity, anonymity, eligibility, individual/universal verifiability, receipt-freeness, and coercion resistance. They demon-

strated that the transparent nature of blockchain transaction ledgers can facilitate voter de-anonymization via network-level traffic analysis and address clustering. Furthermore, they emphasized that client device compromise (malware, screen loggers) is rarely addressed in blockchain literature, and that emerging quantum computing poses long-term threats to classical elliptic curve cryptography (ECDSA secp256k1).

Wermelinger et al. [14] shifted focus from technical architecture to human factors and institutional trust. Utilizing Partial Least Squares Structural Equation Modeling (PLS-SEM) on survey responses from 1,851 Brazilian voters, they demonstrated that systemic institutional trust is the single dominant determinant of electronic voting acceptance and demands for printed voter-verified paper audit trails (VVPAT). Their findings prove that cryptographic security guarantees are insufficient on their own; public confidence demands transparent, understandable verification mechanisms and intuitive user interfaces.

2.5 Smart Contract Approaches and Network Support

Chaudhary et al. [15] deployed an Ethereum smart contract incorporating `addCandidate`, `authorize`, `vote`, and `endElection` routines, optimized for execution over high-speed 5G wireless networks. To mitigate on-chain storage costs, bulky voter verification artifacts were offloaded to the InterPlanetary File System (IPFS), with only IPFS content identifiers (CIDs) committed on-chain. While achieving lower bit-error rates and passing formal Echidna fuzz testing, the protocol retains administrative centralization in contract deployment, and client devices remain vulnerable to local malware.

The foundational baseline for our work is established by Asha et al. [1], who developed a Voting DApp on Ethereum that directly addresses double voting through smart contract logic. In their design, the smart contract maintains state mappings of registered voter addresses and certified candidates. The election administrator retains exclusive authority to invoke `addCandidate` and `registerVoter`. Voters authenticate using MetaMask and cast ballots via a responsive React.js and Web3.js web interface. Their prototype demonstrated successful registration, atomic double-voting rejection, and real-time result tabulation on an Ethereum test network. However, Asha et al. observed that public Ethereum gas price fluctuations and interface

complexity for non-technical users represent operational challenges that necessitate private consortium consensus and optimized user workflows.

2.6 Synthesis and Research Gap Identification

Table 2.1 synthesizes the fifteen surveyed literature works across six core criteria: publication year, blockchain platform, consensus mechanism, authentication methodology, evaluation maturity, and primary technical limitation.

Table 2.1: Systematic Comparative Analysis of Surveyed Blockchain Electronic Voting Literature

Ref.	Year	Platform	Consensus	Authentication	Evaluation	Key Limitation / Bottleneck
[2]	2022	Custom Chain	Pluggable PoW	National ID + SHA-256	Simulation (6×10^6)	Constant connectivity required; no biometrics; 51% attack risk.
[3]	2019	Consortium	Presiding Sig	Cryptographic ID	Conceptual only	No software prototype; heavy administrative centralization.
[4]	2018	Public/Private	Token-transfer	Wallet / Digital Coin	Analytical review	Immature tooling; defect risks; unready for national scale.
[5]	2019	Hyperledger Fabric	Raft / Kafka	e-ID + Fingerprint	Architecture design	No throughput evaluation; heavy hardware needed per booth.
[6]	2025	Ethereum	PoS	Face + Fingerprint + RFID	University election	Expensive biometric hardware; ongoing gas transaction fees.
[7]	2024	Custom PoW	PoW	Facial / Hand Gestures	Trials (N = 100)	Lighting sensitive; finger tracking errors; slow for elderly.
[8]	2021	Hyperledger Fabric	PBFT	ElGamal Blind Signatures	Simulation	Scalability bottleneck under congestion; authority trust.
[9]	2025	Hyperledger Besu	IBFT 2.0	Diff. Privacy + SSI	3-node cluster	Assumes honest authority; requires standardized digital wallets.
[10]	2022	SLR (2010–21)	Diverse	Surveyed taxonomy	Systematic Review	Identifies scalability, coercion, and central eligibility gaps.
[11]	2021	Survey	Diverse	Key management analysis	Comparative Survey	Academic prototypes; lack of standard e-voting protocols.

Continued on next page...

Table 2.1 (Continued): Systematic Comparative Analysis of Surveyed Literature

Ref.	Year	Platform	Consensus	Authentication	Evaluation	Key Limitation / Bottleneck
[12]	2025	Review	PoW / PoS	Web3 / MetaMask	Architectural Review	Highlights gas cost barriers and rural internet connectivity.
[13]	2025	Survey	Diverse	Cryptographic audit	Theoretical Survey	Transparency de-anonymizes voters; client malware ignored.
[14]	2026	Empirical	N/A	PLS-SEM Trust Model	Empirical (N = 1851)	Shows institutional trust dominates over technical guarantees.
[15]	2023	Ethereum	PoW + 5G	Smart Contract + IPFS	Echidna Fuzzing	Client device malware; central contract deployment risks.
[1]	2025	Ethereum	PoA / PoS	MetaMask + RBAC	Remix / Testnet	Public gas volatility; usability for non-technical electorates.

Critical synthesis of the surveyed corpus reveals three persistent research gaps:

1. **Evaluation Gap:** The majority of proposed architectures remain confined to conceptual frameworks or simulated toy elections. Real-world behavior under bursty transaction loads remains unvalidated.

2. **Authentication Vulnerability vs Hardware Overhead:** Existing systems bifurcate into two extremes: wallet-only schemes that are vulnerable to credential theft, and biometric systems that demand prohibitive specialized hardware at every voting precinct.

3. **Consensus and Financial Feasibility Gap:** Deploying on public blockchains (e.g., Ethereum mainnet) imposes volatile, unpredictable gas fees and throughput ceilings (15–30 TPS), whereas permissioned frameworks often re-introduce centralized administrative control.

These identified gaps directly motivate the architecture of our proposed Voting DApp, which enforces atomic double-voting mitigation in Solidity, leverages lightweight MetaMask asymmetric authentication, and employs a consortium Proof-of-Authority (PoA) consensus model to provide predictable, zero-fee, high-throughput voting.

Chapter 3

Problem Definition

A rigorous engineering formulation is essential to transform subjective electoral requirements into verifiable, mathematically bounded smart contract routines. This chapter formalizes the problem statement, defines the threat model and attack vectors, establishes operational boundaries, and details the governing mathematical formulations.

3.1 Problem Statement

In conventional electoral administration, ensuring that every registered voter v exercises their franchise at most once ($\sum \text{ballots}(v) \leq 1$) is enforced through manual ink marking, physical voter rolls, or centralized database records. In decentralized digital voting, voters interact across untrusted networks using pseudonymous cryptographic addresses. The fundamental problem addressed in this project is:

> *To design and implement a decentralized, transparent, and auditable electronic voting architecture that provably prevents duplicate voting (Double-Voting) and unauthorized candidate insertion at the smart contract level, while providing cryptographically non-repudiable voter authentication, real-time result compilation, and deterministic transaction processing without exposing voters to gas cost volatility.*

3.2 Threat Model and Attack Vectors

The proposed system is designed against the following realistic adversarial threats:

- **Double-Voting and Race Conditions:** A malicious voter attempts to cast multiple ballots by executing concurrent asynchronous transactions from the same wallet address or exploiting transaction pool (mempool) propagation latencies.
- **Unauthorized Candidate Insertion and Spoofing:** An unprivileged participant attempts to inject fraudulent candidates or manipulate existing candidate vote counters directly on the blockchain.
- **Sybil and Identity Impersonation Attacks:** An adversary attempts to generate thousands of arbitrary Ethereum wallet addresses to cast illegitimate votes.
- **Transaction Tampering and Forgery:** A man-in-the-middle attacker attempts to modify the selected candidate identifier c while the ballot payload is transmitted from the client browser to the Ethereum network.
- **Post-Election Result Manipulation:** A corrupt election official attempts to retroactively alter recorded tallies or insert uncast ballots into historical blocks.

3.3 System Boundaries and Operational Assumptions

The operational scope is bounded by the following formal assumptions:

1. **Cryptographic Primitives:** The underlying elliptic curve digital signature algorithm (ECDSA on secp256k1) and cryptographic hash functions (SHA-256 and Keccak-256) are computationally secure against classical preimage and collision attacks.
2. **Voter Key Custody:** The voter is responsible for maintaining the physical security of their personal computing device and the private key/seed phrase associated with their MetaMask wallet.
3. **Honest Administrator at Registration:** The election administrator is trusted during the pre-election whitelisting phase to accurately map physical citizen identities to authorized Ethereum wallet addresses. Once whitelisting is completed, the administrator has no computational capability to alter cast votes or influence smart contract execution.

4. **Consensus Integrity:** A majority of validator nodes in the Proof-of-Authority consortium are honest and correctly validate transactions according to the published Solidity contract bytecode.

3.4 Mathematical Formulation and Governing Equations

The security and integrity of the proposed blockchain voting system are formally governed by the following mathematical formulations:

3.4.1 Cryptographic Hash Chaining and Block Integrity

Every block B_i at sequential height i in the distributed ledger is cryptographically bound to its parent block B_{i-1}. The block hash digest $H(B_i)$ is computed as:

$$H(B_i) = \text{SHA-256}\Big(H(B_{i-1}) \;\|\; T_i \;\|\; Tx_i \;\|\; n_i\Big) \tag{3.1}$$

where $H(B_{i-1}) \in \{0,1\}^{256}$ represents the cryptographic hash of the immediate predecessor block, T_i denotes the Unix epoch timestamp, $Tx_i = \{tx_1, tx_2, \ldots, tx_m\}$ represents the Merkle root of the ordered set of verified transactions encapsulated in block i, n_i represents the consensus nonce value, and $\|$ denotes the byte-level string concatenation operator.

Any retroactive attempt to alter a transaction $tx_k \in Tx_j$ in a historical block B_j ($j < i$) produces an altered hash $H'(B_j) \neq H(B_j)$. Because $H(B_j)$ is embedded within block B_{j+1}, this modification breaks the cryptographic link for all subsequent blocks $B_{j+1}, \ldots, B_i$, causing immediate rejection by all network validator nodes.

3.4.2 Atomic Double-Voting Mitigation Predicate

Let $\mathcal{V} \subset \mathbb{Z}_{2^{160}}$ represent the universal set of all 20-byte Ethereum wallet addresses, and let $C = \{1, 2, \ldots, K\}$ represent the set of certified candidate identifiers. The smart contract maintains two internal state mapping functions:

$$\text{registered} : \mathcal{V} \to \{\text{true}, \text{false}\} \tag{3.2}$$

$$\text{voted} : \mathcal{V} \to \{\text{true}, \text{false}\} \tag{3.3}$$

The binary ballot acceptance predicate Accept(v) governing a ballot submission from voter address $v \in \mathcal{V}$ is defined as:

$$\text{Accept}(v) = \begin{cases} 1, & \text{if registered}(v) = \text{true} \ \wedge \ \text{voted}(v) = \text{false} \\ 0, & \text{otherwise} \end{cases} \tag{3.4}$$

When Accept(v) = 1, the smart contract executes an atomic state mutation:

$$\text{voted}(v) \leftarrow \text{true} \tag{3.5}$$

$$N(c) \leftarrow N(c) + 1 \tag{3.6}$$

Because equations (3.5) and (3.6) are executed within a single, atomic EVM transaction context, any secondary transaction submitted by the same address v will encounter voted(v) = true, resulting in Accept(v) = 0 and an immediate EVM transaction revert.

3.4.3 Cumulative Vote Tally and Upper Bound Invariant

The cumulative aggregate vote tally $N(c)$ recorded for any candidate $c \in C$ at the conclusion of polling is formulated as:

$$N(c) = \sum_{v \in \mathcal{V}} \Big[\text{vote}(v) = c \ \wedge \ \text{Accept}(v) = 1 \Big] \tag{3.7}$$

where vote(v) $\in C$ represents the candidate selected by voter v, and $[\cdot]$ denotes the Iverson bracket notation (evaluating to 1 if true, 0 if false).

A critical mathematical invariant enforced by the contract is that the total sum of all candidate votes can never exceed the total number of whitelisted registered voters:

$$\sum_{c \in C} N(c) \ \leq \ \sum_{v \in \mathcal{V}} \Big[\text{registered}(v) = \text{true} \Big] \ \leq \ |\mathcal{V}| \tag{3.8}$$

This invariant mathematically guarantees that ballot fabrication and over-voting are impossible within the decentralized ledger.

Chapter 4

Requirements Specification

The formal System Requirements Specification (SRS) delineates the complete functional capabilities, non-functional quality attributes, and hardware/software execution environments required for the blockchain-based electronic voting system.

4.1 Functional Requirements

The functional requirements are categorized across three operational modules: Administrator Module, Voter Module, and Smart Contract / Ledger Module.

4.1.1 Administrator Module

- **FR-01: Smart Contract Deployment and Ownership:** The election administrator shall deploy the compiled Solidity contract bytecode to the Ethereum network, automatically registering the deployer's address as the immutable `admin` authority.
- **FR-02: Candidate Management:** The administrator shall have exclusive authority to register certified candidate monikers via `addCandidate(string memory name)`. The system shall automatically assign sequential candidate IDs and initialize vote tallies to zero.
- **FR-03: Voter Whitelisting:** The administrator shall have exclusive authority to whitelist eligible voter Ethereum addresses via `registerVoter(address _voterAddress)`.

- **FR-04: Role-Based Authorization Guard:** Any attempt by a non-administrator address to invoke administrative functions shall result in an immediate smart contract revert.

4.1.2 Voter Module

- **FR-05: Web3 Wallet Handshake:** The voter interface shall interface with the `window.ethereum` API provided by MetaMask, prompting the voter to authorize connection and retrieve their active account address.
- **FR-06: Eligibility Verification:** Upon wallet connection, the front end shall query the smart contract state to verify that the connected address is registered and has not previously voted.
- **FR-07: Certified Candidate Display:** The front end shall dynamically fetch the certified candidate registry from the blockchain and display candidate profiles to authorized voters.
- **FR-08: Ballot Transaction Construction and Signing:** Upon selecting a candidate, the voter shall trigger a cryptographically signed transaction via MetaMask, encoding the function invocation `vote(uint _candidateId)`.

4.1.3 Smart Contract and Ledger Module

- **FR-09: Atomic Double-Voting Mitigation:** The smart contract shall verify that the caller is registered and has not voted. If verified, it shall atomically set the voter's participation flag to `true` and increment the candidate's tally.
- **FR-10: State Reversion on Violation:** If an unregistered address attempts to vote, or if a registered voter attempts to vote a second time, the contract shall immediately revert with an explicit error string.
- **FR-11: Event Emission:** Upon successful ballot commitment, the smart contract shall emit a public `VoteCast(address indexed voter, uint indexed candidateId, uint timestamp)` event.

- **FR-12: Public Audit Query:** The contract shall expose public, gas-free read functions (`candidatesCount`, `candidates`, `totalVotes`) enabling continuous external auditing.

4.2 Non-Functional Requirements

- **NFR-01: Immutability and Tamper-Resistance:** All confirmed votes and administrative actions shall be permanently recorded on an append-only distributed ledger, rendering retroactive alteration computationally impossible.
- **NFR-02: Transaction Latency and Throughput:** In a Proof-of-Authority consortium configuration, average block sealing latency shall not exceed 2.0 seconds, supporting peak burst throughput of at least 100 transactions per second.
- **NFR-03: Zero-Gas Cost for Voters:** In consortium deployment, transaction gas price shall be configured to zero ($G_{\text{price}} = 0$) or subsidized, ensuring voters incur no financial cost to exercise their franchise.
- **NFR-04: Non-Repudiation:** Every transaction shall carry a valid ECDSA signature generated by the voter's private key, guaranteeing that vote submissions cannot be denied by the sender.
- **NFR-05: Cross-Browser Accessibility:** The web client shall operate seamlessly across all major Web3-compatible browsers, including Google Chrome, Mozilla Firefox, Brave, and Microsoft Edge.

4.3 Hardware and Software Specifications

Table 4.1 outlines the operational hardware and software environments utilized for system development, testbed deployment, and client execution.

Table 4.1: System Hardware and Software Technical Specifications

Category	Component / Tool	Specification / Version
Blockchain Core & Consensus Layer	Blockchain Runtime	Ethereum Virtual Machine (EVM) / Go-Ethereum (Geth v1.13+)
	Consensus Protocol	Proof-of-Authority (PoA / Clique Engine, IBFT 2.0)
	Smart Contract Language	Solidity (v0.8.20+ with automated overflow checking)
	Local Testbed Network	Ganache CLI / Hardhat In-Memory Network
	Development Toolchain	Hardhat Suite (v2.19+) / Truffle / Remix IDE
Middleware & Application Layer	Front-End Architecture	React.js (v18.2+) / Vite Engine / HTML5 / CSS3
	Web3 Provider Bridge	Web3.js (v1.10+) / Ethers.js (v6.x) JSON-RPC Client
	Cryptographic Wallet	MetaMask Browser Extension (v11.x+)
	RPC Communication	JSON-RPC 2.0 over Secure WebSocket (WSS) and HTTP
	Decentralized Storage	InterPlanetary File System (IPFS) / Pinata Gateway
Validator & Server Hardware	Central Processor (CPU)	Intel Core i7 / AMD Ryzen 7 (8 Cores, 16 Threads @ 3.6 GHz)
	System Memory (RAM)	16 GB / 32 GB DDR4 3200 MHz High-Speed RAM
	Primary Storage	512 GB NVMe M.2 Solid State Drive (Read $\geq$ 3500 MB/s)
	Host Operating System	Ubuntu Server 22.04 LTS (64-bit Linux Kernel 5.15+)
	Network Interface	Gigabit Ethernet (1000BASE-T Full-Duplex Dedicated Link)
Client Execution Environment	Supported Platforms	Commodity Desktop PC, Laptop, Smartphone, Tablet
	Client Operating Systems	Windows 10/11, macOS (v12+), Linux (Ubuntu), Android, iOS
	Compatible Web Browsers	Google Chrome (v115+), Mozilla Firefox, Brave, MS Edge
	Network Bandwidth	Broadband / 4G / 5G Low-Latency Connection ($\geq$ 2 Mbps)
	Display Layout	Responsive UI (360 $\times$ 640 px Mobile to 1080p Desktop)

Chapter 5

System Design

This chapter details the architectural decomposition, component interactions, workflows, formal algorithms, block structures, relational state schemas, and Data Flow Diagrams (DFDs) governing the decentralized electronic voting system.

5.1 Multi-Tier System Architecture

The system architecture is structured across four decoupled, modular tiers: Client Presentation Tier, Web3 Middleware Tier, Smart Contract Logic Tier, and Blockchain Ledger Tier. Figure 5.1 illustrates this multi-tier architecture and the communication pathways between components.

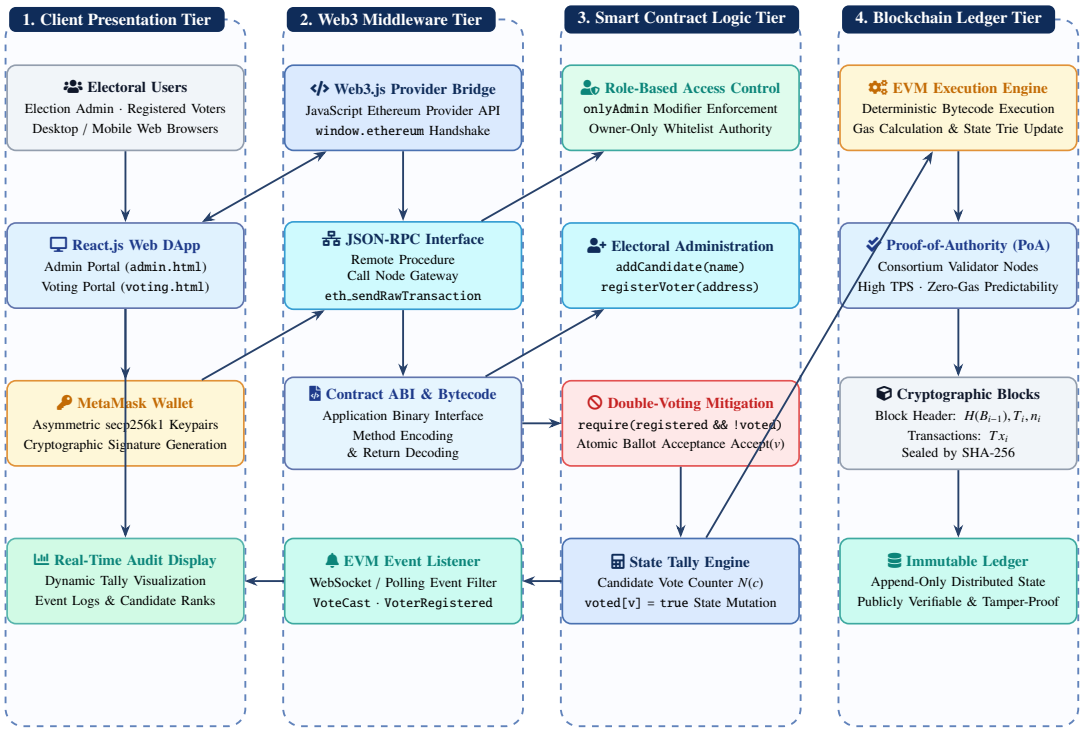


Figure 5.1: Multi-tier system architecture of the blockchain-based electronic voting decentralized application (Voting DApp).

The Client Presentation Tier provides distinct portals for the election administrator (`admin.html`) and voters (`voting.html`). Voters authenticate via MetaMask, which manages private key storage and signs transactions locally. The Web3 Middleware Tier translates client actions into JSON-RPC calls sent to the Ethereum network. The Smart Contract Logic Tier executes the compiled Solidity bytecode on the EVM, enforcing role-based access control and atomic double-voting mitigation. Finally, the Blockchain Ledger Tier seals verified transactions into chronologically chained blocks via Proof-of-Authority consensus.

5.2 Proposed Operational Methodology Pipeline

The complete election lifecycle is organized into five sequential phases, as illustrated in Figure 5.2.

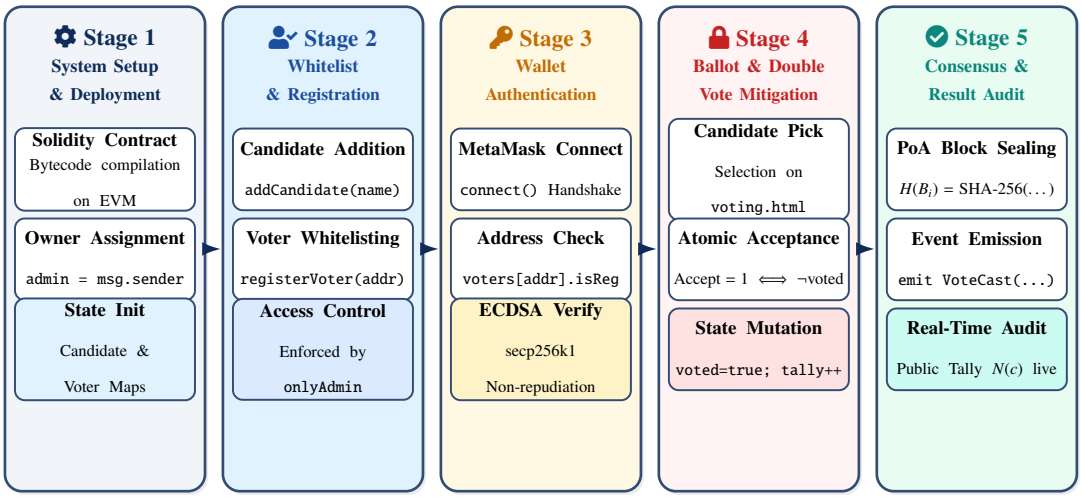


Figure 5.2: Proposed operational methodology pipeline detailing the five stages of the election lifecycle.

- **Stage 1 (System Setup & Contract Deployment):** The administrator compiles and deploys the Solidity smart contract, establishing contract ownership and initializing the candidate and voter state mappings.
- **Stage 2 (Entity Whitelisting & Registration):** The administrator registers certified candidates via `addCandidate` and whitelists eligible voter public wallet addresses via `registerVoter`.
- **Stage 3 (Wallet Authentication & Handshake):** The voter visits the portal, initiates a connection via `window.ethereum`, and MetaMask returns the voter's public address without exposing the underlying private key.
- **Stage 4 (Ballot Submission & Double-Voting Mitigation):** The voter selects a candidate and signs the transaction. The smart contract evaluates the atomic condition Accept(v); if valid, it sets voted(v) = true and increments $N(c)$.
- **Stage 5 (Consensus Sealing & Live Audit Compilation):** Validator nodes seal the transaction into a block via SHA-256 hash chaining, emit the `VoteCast` event, and update the live audit tally.

5.3 Voter Registration and Whitelisting Workflow

Figure 5.3 illustrates the step-by-step handshake between the administrator, voter, MetaMask wallet, and the smart contract during the pre-voting whitelisting and

authentication phases.

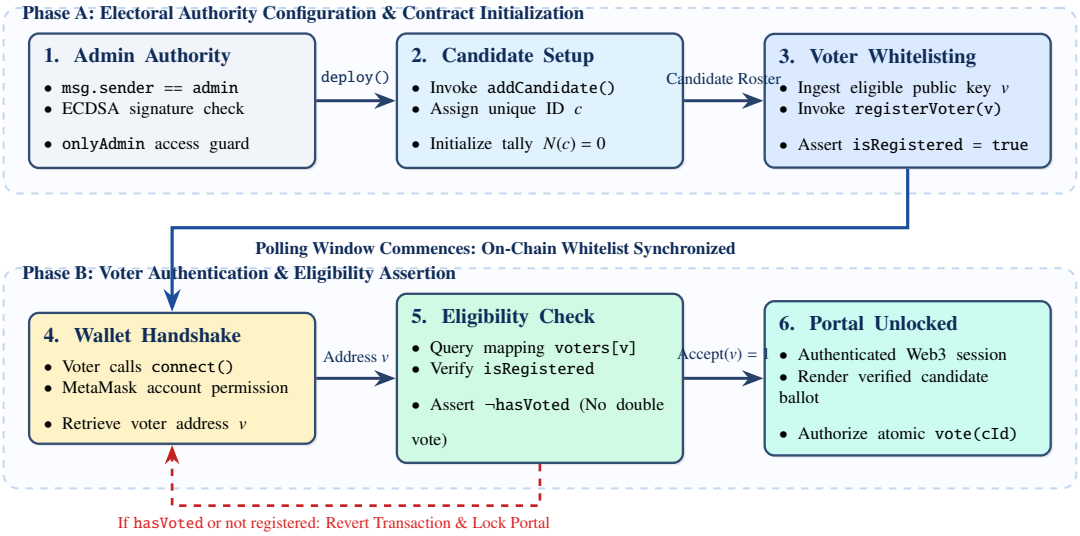


Figure 5.3: Detailed layered workflow for voter whitelisting, MetaMask connection, and eligibility verification.

5.4 End-to-End Ballot Sequence Flow

Figure 5.4 illustrates the end-to-end transaction lifecycle across the four interacting system entities: the Voter & MetaMask Client, the React & Web3.js Middleware, the Smart Contract on the EVM, and the Consensus Ledger.

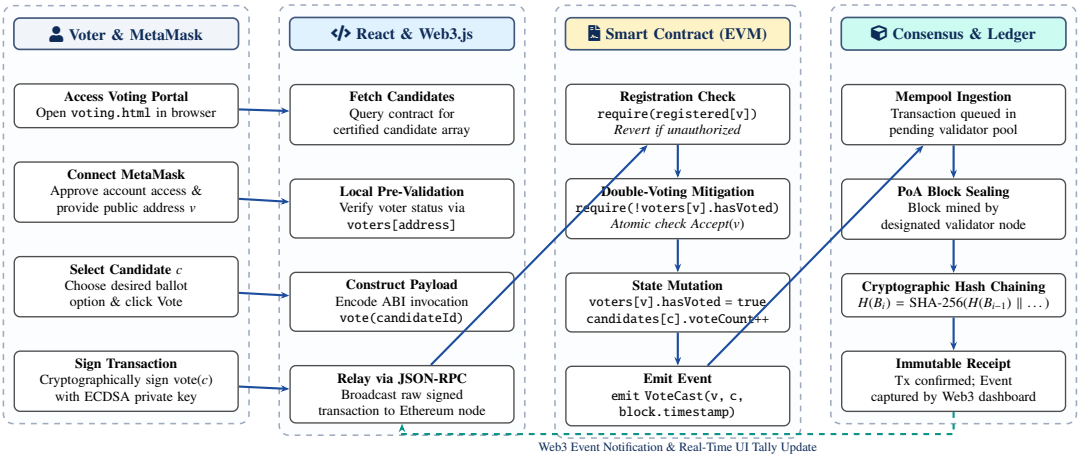


Figure 5.4: End-to-end system sequence flow from ballot selection to block sealing and real-time Web3 tally notification.

5.5 Complete System Operational Flowchart

Figure 5.5 depicts the complete operational flowchart of the voting decentralized application, detailing role verification, whitelisting validation, double-voting rejection branches, and event emissions.

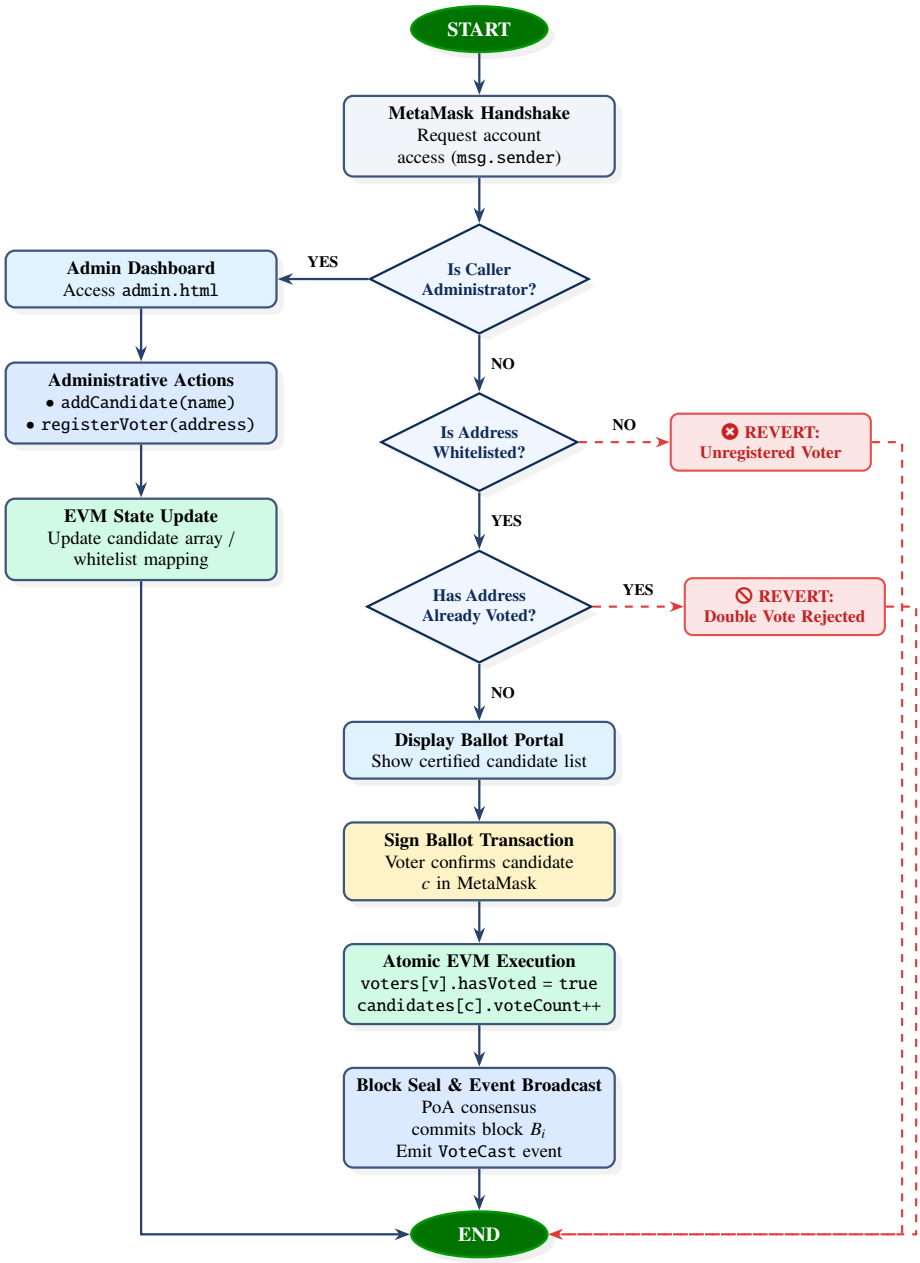


Figure 5.5: Comprehensive system flowchart detailing administrator actions, voter verification branches, and atomic double-voting mitigation.

5.6 Algorithm

The complete operational mechanics of the blockchain-based electronic voting system and its double-voting mitigation engine are formalized through the end-to-end procedural steps detailed below:

Step 1: System Initialization and Contract Deployment

1.1 Compile and deploy smart contract bytecode:

1.1.1 Administrator compiles Solidity smart contract bytecode using the compiler `solc` (version $\geq$ 0.8.20).

1.1.2 Administrator deploys contract deployment transaction to the Ethereum network via the authenticated JSON-RPC endpoint.

1.2 Execute atomic constructor initialization:

$$\text{admin} \leftarrow \text{msg.sender}, \quad \text{candidatesCount} \leftarrow 0, \quad \text{electionOpen} \leftarrow \text{true} \tag{5.1}$$

1.2.1 Contract constructor binds administrative ownership to deployer wallet address and initializes zeroed ballot state.

1.2.2 Contract emits `ElectionInitialized(admin, block.timestamp)` event log to the blockchain ledger.

Step 2: Candidate Nomination and Registration

2.1 Submit candidate nomination and evaluate administrative authority:

2.1.1 Administrator invokes addCandidate(name) passing the certified candidate moniker string.

2.1.2 Contract evaluates caller authorization modifier barrier: **assert**(msg.sender == admin).

2.2 Persist candidate state struct to on-chain storage:

$$\begin{gathered} \text{candidatesCount} \leftarrow \text{candidatesCount} + 1 \\ \text{candidates[candidatesCount]} \leftarrow \text{Candidate}(\text{candidatesCount}, \text{name}, 0) \end{gathered} \tag{5.2}$$

2.2.1 Candidate record is appended to the persistent mapping with initialized zero vote tally.

2.2.2 Contract emits on-chain event: `CandidateAdded(candidatesCount, name)`.

Step 3: Voter Whitelisting and Address Certification

3.1 Ingest verified identity credentials:

3.1.1 Administrator ingests eligible citizen's verified 20-byte Ethereum wallet address $v \in \mathcal{V}$.

3.1.2 Administrator initiates whitelisting transaction via registerVoter(v).

3.2 Evaluate access barrier and instantiate voter struct:

$$\text{voters}[v] \leftarrow \text{Voter}(v, \text{isRegistered} = \text{true}, \text{hasVoted} = \text{false}, \text{candidateId} = 0) \quad (5.3)$$

3.2.1 Contract enforces access restriction: **assert**(msg.sender == admin).

3.2.2 Contract verifies unwhitelisted state invariant: **assert**(voters[v].isRegistered == false).

3.2.3 Contract records voter struct and emits audit receipt: `VoterRegistered(v, block.timestamp)`.

Step 4: Client Initialization and Wallet Handshake

4.1 Discover injected Ethereum provider:

4.1.1 Voter navigates to web portal (`voting.html`) using a Web3-compatible browser.

4.1.2 Client script validates provider injection: **assert**(window.ethereum $\neq$ null).

4.2 Perform asynchronous account authorization handshake:

4.2.1 Voter triggers interface connection routine; client initiates RPC procedure `eth_requestAccounts`.

4.2.2 MetaMask prompts voter for access consent and returns active account address v = accounts[0].

Step 5: Client-Side Eligibility Verification

5.1 Query on-chain voter registration state:

5.1.1 Front end instantiates Web3 contract wrapper using deployed contract address and JSON ABI.

5.1.2 Front end executes read-only query call:

$$\text{voterRecord} \leftarrow \text{contract.methods.voters}(v).\text{call}() \tag{5.4}$$

5.2 Enforce client gatekeeping logic:

5.2.1 If voterRecord.isRegistered == false, UI displays "Access Denied: Unregistered Voter".

5.2.2 If voterRecord.hasVoted == true, UI displays "Ballot Already Cast: Double Voting Prohibited".

5.2.3 If voterRecord.isRegistered == true $\wedge$ voterRecord.hasVoted == false, UI unlocks certified candidate ballot selection view.

Step 6: Dynamic Candidate Presentation

6.1 Fetch candidate records from contract:

6.1.1 Front end queries candidate registry size: candidatesCount $\leftarrow$ contract.methods.candidatesCount().call().

6.1.2 For each candidate identifier $c \in [1, \text{candidatesCount}]$, front end queries candidate record:

$$\text{candidate}[c] \leftarrow \text{contract.methods.candidates}(c).\text{call}() \tag{5.5}$$

6.2 Dynamically render interactive ballot portal:

6.2.1 Front end dynamically renders candidate radio button list containing names, IDs, and candidate badges.

Step 7: Ballot Selection and Transaction Serialization

7.1 Serialize ABI-encoded vote call:

7.1.1 Voter selects desired candidate identifier $c \in C$ and confirms ballot submission.

7.1.2 Web3.js serializes function call payload: data $\leftarrow$ encodeABI(vote(c)).

7.2 Estimate transaction computational gas:

$$G_{\text{limit}} \leftarrow \text{contract.methods.vote}(c).\text{estimateGas}(\{\text{from} : v\}) \tag{5.6}$$

7.2.1 Gas parameters are assembled into transaction payload alongside active network

gas price.

Step 8: Asymmetric ECDSA Transaction Signing

8.1 Compute Keccak-256 transaction digest:

$$h_{\text{tx}} \leftarrow \text{Keccak-256}(\text{nonce} \parallel G_{\text{price}} \parallel G_{\text{limit}} \parallel \text{to} \parallel \text{value} \parallel \text{data}) \tag{5.7}$$

8.1.1 Front end transmits unsigned transaction parameters to MetaMask wallet via JSON-RPC.

8.2 Sign transaction and broadcast to peer network:

$$\sigma = (r, s, v) \leftarrow \text{ECDSA-Sign}(h_{\text{tx}}, K_{\text{priv}}) \tag{5.8}$$

8.2.1 Voter reviews gas estimate and clicks "Confirm" in wallet modal.

8.2.2 MetaMask generates elliptic curve signature σ using voter's private key K_{priv} without exposing private keys.

8.2.3 Client broadcasts signed raw transaction via RPC procedure `eth_sendRawTransaction`.

Step 9: Smart Contract Execution and Atomic Double-Voting Mitigation

9.1 Recover sender address and enforce cryptographic guard conditions:

9.1.1 EVM recovers public address: $v \leftarrow \text{ecrecover}(h_{\text{tx}}, v, r, s)$.

9.1.2 Contract enforces whitelist guard: **require**(voters[v].isRegistered == true, "Voter is not registered").

9.1.3 Contract enforces atomic double-voting barrier: **require**(voters[v].hasVoted == false).

9.1.4 Contract enforces candidate boundary check: **require**($c \geq 1 \wedge c \leq$ candidatesCount, "Invalid candidate selected").

9.2 Execute atomic on-chain state mutation:

$$\begin{gathered}\text{voters}[v].\text{hasVoted} \leftarrow \text{true}, \quad \text{voters}[v].\text{votedCandidateId} \leftarrow c \\ \text{candidates}[c].\text{voteCount} \leftarrow \text{candidates}[c].\text{voteCount} + 1\end{gathered} \tag{5.9}$$

9.2.1 Double-voting status flag is set atomically in the same transaction execution

context.

9.2.2 Contract emits event log: emit VoteCast(v, c, block.timestamp).

Step 10: Consensus, Block Sealing, and Ledger Commitment

10.1 Mempool queuing and candidate block packaging:

10.1.1 Transaction enters validator mempool and is packaged into block candidate B_i.

10.1.2 Designated Proof-of-Authority validator signs block header with designated validator private key.

10.2 Compute cryptographic hash chaining and broadcast block:

$$H(B_i) \leftarrow \text{SHA-256}(H(B_{i-1}) \parallel T_i \parallel Tx_i \parallel n_i) \tag{5.10}$$

10.2.1 Cryptographic block hash is computed and linked to prior block hash $H(B_{i-1})$.

10.2.2 Block B_i is distributed across P2P validator network; ledger state root is permanently committed.

Step 11: Real-Time Event Propagation and UI Notification

11.1 Capture WebSocket event notifications:

11.1.1 Web3.js WebSocket provider captures emitted `VoteCast` event log.

11.1.2 Front end parses event receipt: verifies transaction hash and confirmation block number.

11.2 Update user interface state:

11.2.1 UI dynamically updates to confirm successful ballot casting and renders immutable cryptographic transaction receipt.

Step 12: Polling Closure and Certified Result Tabulation

12.1 Conclude election session:

12.1.1 When polling window closes, administrator invokes closeElection() on the smart contract.

12.2 Tabulate certified verifiable results:

$$N(c) = \text{candidates}[c].\text{voteCount}, \quad \forall c \in [1, \text{candidatesCount}] \tag{5.11}$$

12.2.1 Final tallies are computed directly from the immutable chain state.

12.2.2 Certified results are displayed on the public audit dashboard; mathematical invariant $\sum N(c) \leq |\mathcal{V}|$ holds.

5.7 Block Structure and Cryptographic Chaining

Figure 5.6 details the internal architecture of blocks within the blockchain ledger, demonstrating how block headers, cryptographic timestamps T_i, nonces n_i, and transaction sets Tx_i are chained via SHA-256 hashing.

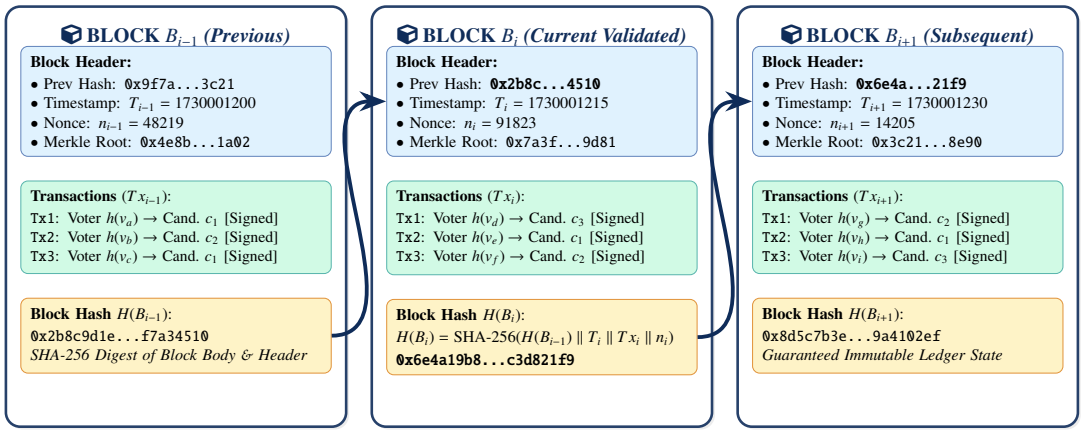


Figure 5.6: Cryptographic block structure and hash chaining mechanism illustrating ledger immutability.

The cryptographic linking between consecutive blocks B_{i-1} and B_i guarantees that historical records cannot be retroactively modified without invalidating all downstream block hashes. Each block encapsulates the Merkle root summarizing its transaction set Tx_i, ensuring that any attempted alteration of a single cast vote invalidates the entire block header digest.

5.8 Smart Contract State Schemas and Structs

Figure 5.7 depicts the relational entity connections, state mapping layouts, and event logging structures organized across three primary architectural tiers in the Solidity smart contract.

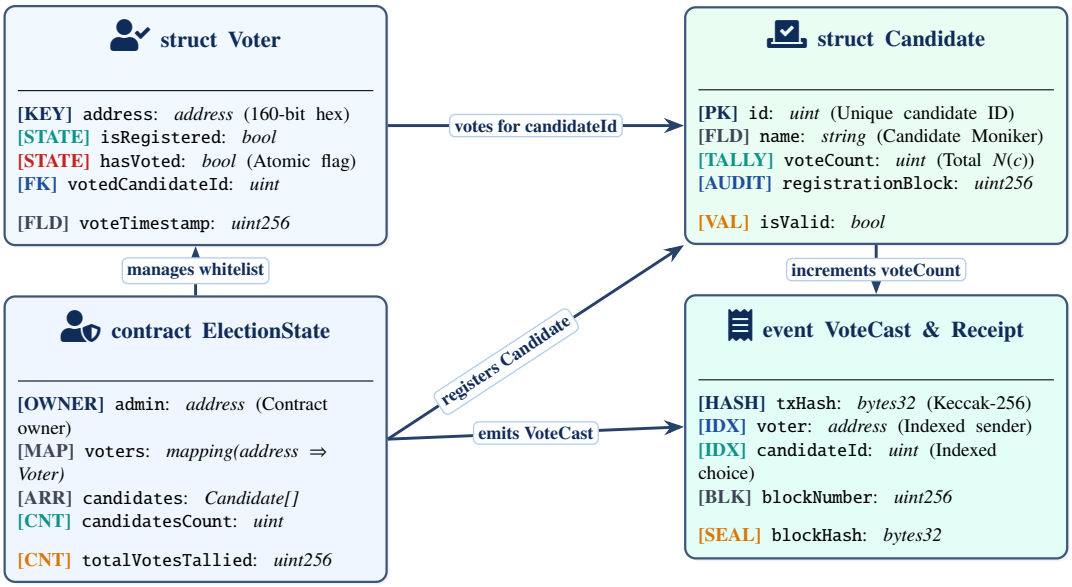


Figure 5.7: Smart contract state schemas, data structs, and relational mappings governing voters and candidates.

Architectural State Decomposition: The smart contract decouples voter authorization, candidate registry, and audit logging into three distinct operational layers:

- **Entity Structs:** `struct Voter` enforces single-ballot casting via the atomic boolean flag `hasVoted` ($O(1)$ duplicate prevention), while `struct Candidate` maintains verified identity profiles and live vote tallies $N(c)$.
- **Relational Storage Mappings:** `mapping(address => Voter) public voters` provides constant-time lookup during ballot validation while safeguarding voter roster privacy. The `Candidate[] public candidates` array enables dynamic candidate onboarding and bounds checking.
- **Access Control and Event Emission:** Privileged administrative actions are restricted to `address public admin` via the `onlyAdmin` modifier. Each accepted ballot atomically increments $N(c)$ and emits a standardized `VoteCast` event into the EVM receipt trie for real-time, gasless auditability by external Web3 observers.

5.9 Level-1 Data Flow Diagram (DFD)

Figure 5.8 illustrates the Level-1 Data Flow Diagram (DFD) characterizing the operational data pipelines, external actor interfaces, smart contract execution routines, and persistent ledger data stores.

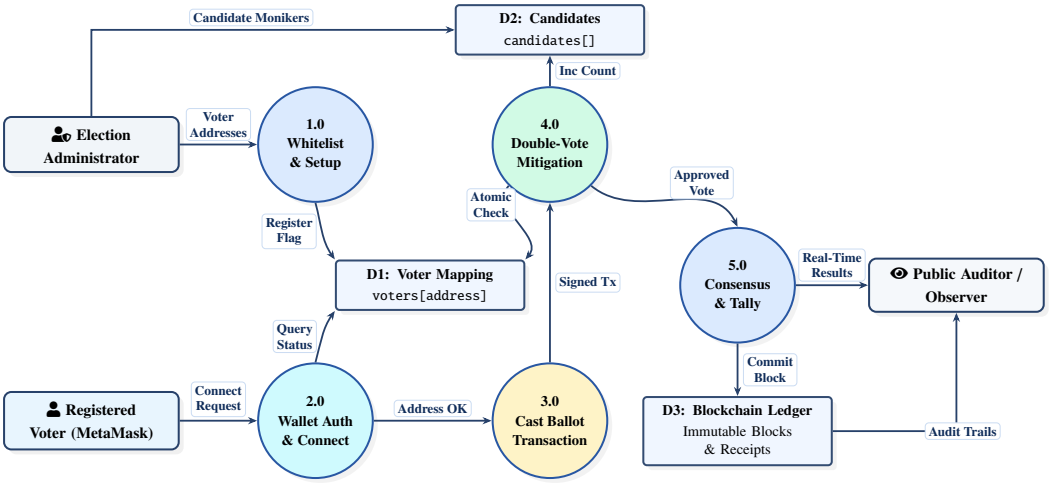


Figure 5.8: Level-1 Data Flow Diagram (DFD) for the blockchain-based electronic voting system.

Process Functional Decomposition: The core electoral operational logic is partitioned into five distinct transformation processes:

- **Process 1.0 (Whitelist & Setup):** Ingests verified candidate monikers and eligible voter public keys from the *Election Administrator*, writing initialized candidate records to **D2 (Candidates)** and authorization flags to **D1 (Voter Mapping)**.
- **Process 2.0 (Wallet Auth & Connect):** Receives the `window.ethereum` handshake from the *Registered Voter*, queries **D1** to verify eligibility, and returns address confirmation without exposing client private keys.
- **Process 3.0 (Cast Ballot Transaction):** Formulates the ABI-encoded transaction payload with candidate choice c, coordinates ECDSA signing via MetaMask, and broadcasts the signed transaction to the network.
- **Process 4.0 (Double-Vote Mitigation):** Executes within an atomic EVM transaction context. It evaluates the predicate Accept(v) against **D1**; if valid, it atomically sets voted(v) $\leftarrow$ true and increments the candidate tally in **D2**.
- **Process 5.0 (Consensus & Tally Compilation):** Commits verified transactions into chained PoA blocks on **D3 (Blockchain Ledger)**, emitting `VoteCast` receipts and live tallies to the *Public Auditor / Observer*.

Data Store Specifications: The architecture enforces strict separation across three persistent data stores:

- **D1: Voter Mapping (`voters[address]`):** Internal key-value mapping storing registration validity, participation flags, and block timestamps. Write access is restricted to administrator setup and the atomic vote execution guard.
- **D2: Candidates Registry (`candidates[]`):** Dynamically sized array storing certified candidate profiles, monikers, and real-time accumulated vote totals $N(c)$.
- **D3: Blockchain Ledger (Immutable Blocks & Receipts):** The append-only distributed ledger containing cryptographic block headers, state roots, receipts, and immutable transaction logs.

Data Flow Dictionary and State Invariants: Table 5.1 specifies the input-output parameters, directional endpoints, and mathematical state invariants enforced across each data flow illustrated in Figure 5.8.

Flow	Source Entity	Target Destination	Transmitted Data Payload	Integrity / State Invariant
F_1	Administrator	D2: Candidates	Candidate names, registration ID	Only owner can add candidates
F_2	Administrator	D1: Voter Mapping	Public wallet addresses ($\mathcal{V}$)	isRegistered(v) $\leftarrow$ true
F_3	Registered Voter	P2.0: Auth	Web3 handshake request	Validates active network ID
F_4	Registered Voter	P3.0: Cast Ballot	Selected Candidate ID c	$c \in [1, \text{candidatesCount}]$
F_5	P3.0	P4.0: Mitigation	Signed ECDSA raw tx	voted(v) == false
F_6	P4.0	D1 & D2	State mutation vectors	Atomic commit (voted $\leftarrow$ true)
F_7	P5.0	D3: Ledger	Sealed block header	SHA-256 hash pointer linkage
F_8	D3 / P5.0	Public Auditor	Transaction receipts, logs	$\sum N(c) \leq \lvert\mathcal{V}\rvert$

Table 5.1: Data Flow Dictionary detailing communication flows and integrity constraints.

Architectural Security and Trust Assumptions: The operational integrity of the data flow pipeline rests upon three foundational guarantees:

1. **Atomicity of Ballot Ingestion:** Because Process 4.0 evaluates the voter's participation record and mutates state within a single EVM execution frame, race conditions and concurrent re-entrancy attempts are structurally impossible.
2. **Cryptographic Non-Repudiation:** External actors cannot forge ballots because every transaction payload received by Process 3.0 must contain an ECDSA signature generated by the voter's private key held securely in MetaMask.
3. **Public Auditability Without Compromising Ballot Privacy:** While the public auditor monitors tallies and verifies block proofs through Process 5.0, individual candidate choices cannot be correlated with off-chain legal identities.

Chapter 6

Design and Methodology

This chapter expounds the mathematical foundations, Solidity smart contract engineering, staged transaction verification, React/Web3.js front-end integration, MetaMask cryptographic signing, and consensus model trade-offs.

6.1 Solidity Smart Contract Architecture

The core electoral logic is implemented in Solidity (version 0.8.20). Solidity provides strong typing, native cryptographic primitives, and built-in overflow/underflow protection, preventing integer wrap-around attacks without external libraries.

The smart contract defines two primary composite data structures:

- `struct Candidate`: Encapsulates a unique candidate identifier (`uint id`), moniker string (`string name`), and accumulated vote tally (`uint voteCount`).
- `struct Voter`: Encapsulates the voter's whitelisting eligibility status (`bool isRegistered`), participation status (`bool hasVoted`), candidate identifier voted for (`uint votedCandidateId`), and the block timestamp of participation.

State mappings link addresses to records:

```
mapping(address => Voter) public voters;
Candidate[] public candidates;
address public admin;
uint public candidatesCount;
```

Administrative privilege separation is enforced through a custom function modifier:

```
modifier onlyAdmin() {
    require(
        msg.sender == admin,
        "Caller is not certified admin"
    );
    _;
}
```

Any invocation of `addCandidate` or `registerVoter` from an address other than `admin` triggers an immediate EVM revert, consuming minimal gas and preventing unauthorized access.

6.2 Staged Execution and Double-Voting Mitigation Pipeline

Figure 6.1 illustrates the staged execution pipeline enforced by the EVM when processing an incoming ballot transaction.

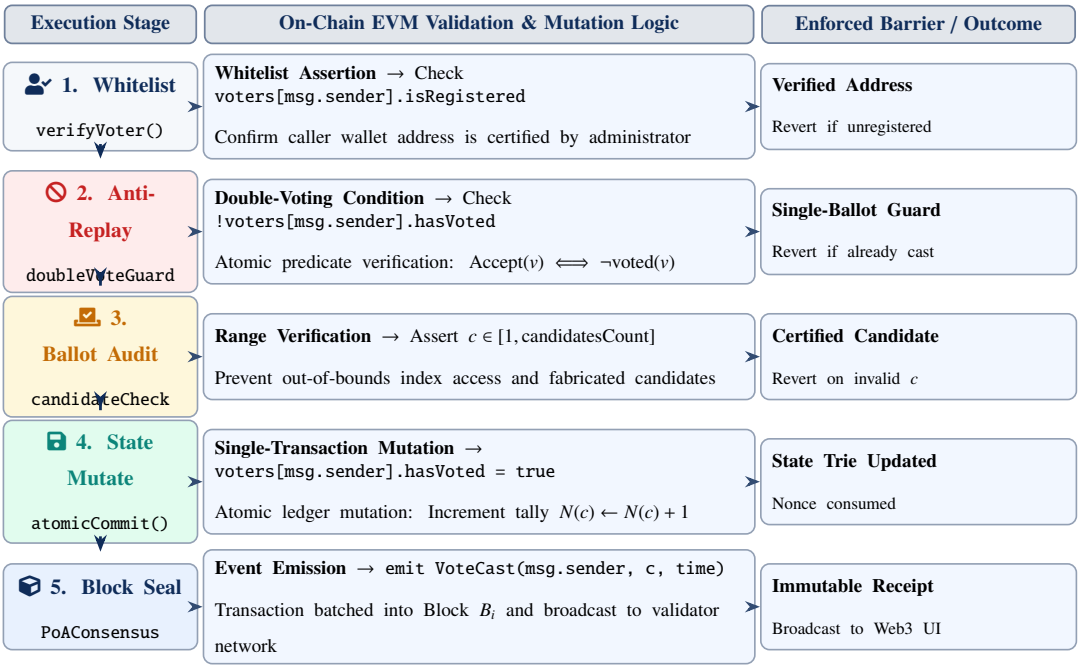


Figure 6.1: Staged execution flow and atomic double-voting mitigation pipeline within the EVM runtime.

The pipeline executes five strict assertions in sequential order:

1. **Whitelist Assertion:** `require(voters[msg.sender].isRegistered, "Voter is not registered");` verifies that the voter's public address was authorized by the administrator during Stage 2.
2. **Double-Voting Guard:** `require(!voters[msg.sender].hasVoted, "Voter has already cast a ballot");` checks that the participation flag is false.
3. **Ballot Range Check:** `require(_candidateId >= 1 && _candidateId <= candidatesCount, "Invalid candidate ID");` ensures that the selected candidate exists in the candidate registry.
4. **Atomic State Mutation:** The contract updates the state atomically by setting `hasVoted = true` and incrementing `candidates[_candidateId].voteCount++`.
5. **Event Emission:** The contract emits the `VoteCast` event, notifying external Web3 listeners.

6.3 Front-End DApp Architecture with React.js and Web3.js

The front end is engineered using React.js, providing a reactive single-page interface that communicates with the blockchain via Web3.js. Web3.js serves as an abstraction layer over the Ethereum JSON-RPC protocol.

When the application loads, it detects the injected Ethereum provider:

```
if (window.ethereum) {
    const web3 = new Web3(window.ethereum);
    await window.ethereum.request({ method: 'eth_requestAccounts' });
    const accounts = await web3.eth.getAccounts();
    setUserAddress(accounts[0]);
}
```

Event listeners are attached to `window.ethereum.on('accountsChanged')` and `window.ethereum.on('chainChanged')`, ensuring that if a user switches their MetaMask account or network, the front-end dynamically refreshes voter eligibility status without requiring a manual browser reload.

6.4 MetaMask Cryptographic Handshake and Non-Repudiation

MetaMask acts as an isolated cryptographic keychain. When a voter submits a ballot, the front end does not have access to the voter's private key. Instead, the transaction payload is handed to MetaMask, which requests explicit user authorization via a popup window.

The transaction is signed using the Elliptic Curve Digital Signature Algorithm (ECDSA) on the secp256k1 curve:

$$y^2 = (x^3 + 7) \pmod{p} \tag{6.1}$$

The signature tuple (r, s, v) guarantees two fundamental security properties:

- **Integrity:** Any modification of the transaction parameters (recipient address, candidate ID, nonce) invalidates the signature.
- **Non-Repudiation:** Only the entity holding the private key corresponding to the registered address could have generated the signature.

6.5 Consensus Mechanism Analysis

The choice of consensus mechanism directly dictates throughput, latency, energy consumption, and financial viability:

- **Proof-of-Work (PoW):** Provides high security against Sybil attacks through hash power competition, but suffers from low throughput (15 TPS), high latency (12–15 seconds block time), and massive electrical waste.
- **Proof-of-Stake (PoS):** Replaces miners with capital validators, drastically reducing power consumption and improving throughput. However, public PoS networks (like Ethereum mainnet) still expose users to variable gas fees driven by market congestion.
- **Proof-of-Authority (PoA):** Operates on a permissioned network where pre-defined, vetted institutional entities (e.g., universities, election commissions,

judiciary representatives) act as designated block sealers. PoA eliminates computational waste, achieves block times of 1–2 seconds, supports 100+ TPS, and enables zero-gas transactions ($G_{\text{price}} = 0$).

For an institutional or national election with a defined polling window, a consortium Proof-of-Authority architecture represents the optimal balance of speed, zero financial cost to voters, and decentralized oversight among independent institutional validators.

Chapter 7

Preliminary Analysis

This chapter presents a comparative evaluation of blockchain platforms, consensus models, EVM gas expenditures, and smart contract vulnerability analysis.

7.1 Comparative Platform Evaluation

Selecting an optimal blockchain framework requires evaluating throughput, smart contract capabilities, development tooling, and decentralization trade-offs. Table 7.1 compares four leading candidate frameworks.

Table 7.1: Comparative Analysis of Leading Blockchain Platforms for Electronic Voting

Platform	Consensus	Throughput (TPS)	Suitability for E-Voting
Ethereum (PoA/Geth)	Clique / IBFT 2.0	100 – 400	**High:** Robust EVM tooling, mature Web3/MetaMask integration, zero-gas support.
Ethereum (Public PoS)	LMD-GHOST / Casper	15 – 30	**Moderate:** Maximum decentralization, but volatile gas fees and throughput caps.
Hyperledger Fabric	Raft / PBFT	500 – 2,000	**Moderate:** High enterprise throughput, but complex setup and lacks web3 wallet ubiquity.
Polygon (PoS / L2)	Bor (PoS) + Heimdall	1,000 – 7,000	**High:** EVM-compatible Layer-2 scaling, low gas fees, but relies on bridge security.

7.2 Consensus Models Comparison

Table 7.2 evaluates consensus models across critical electoral criteria.

Table 7.2: Consensus Mechanism Evaluation Across Performance and Security Metrics

Consensus Model	**Block Latency**	**Energy Usage**	**51% Fault Tolerance**	**Gas Cost Volatility**
Proof-of-Work	12 – 15 s	Extreme (> TWh)	Computational power	High (Market-driven)
Proof-of-Stake	12 s (slots)	Very Low	Stake weight (33% / 67%)	High (Market-driven)
Proof-of-Authority	1 – 2 s	Negligible	Validator collusion (> 50%)	Zero / Predictable
PBFT	< 1 s	Negligible	Byzantine faults (> 33%)	Zero (Permissioned)

7.3 Gas Consumption and Cost Analysis

In Ethereum-based networks, every computational opcode consumes a deterministic amount of gas. Table 7.3 provides an empirical breakdown of gas consumption measured during local testbed execution.

Table 7.3: Empirical Smart Contract Gas Consumption Analysis

Smart Contract Function	**Gas Consumed (Units)**	**Primary Opcode Costs**	**Execution Frequency**
`Contract Deployment`	482,190	`CREATE`, `CODECOPY`	Once per election
`addCandidate(name)`	46,215	`SSTORE` (Candidate array), `MSTORE`	Once per candidate
`registerVoter(address)`	28,450	`SSTORE` (Whitelist mapping)	Once per eligible voter
`vote(candidateId)`	44,180	`SLOAD`, `SSTORE` (×2), `LOG2`	Once per participating voter
`getResults() [View]`	0 (Gas-Free)	`STATICCALL`, `SLOAD`	Unlimited (Public audits)

On a public network with a gas price of 25 Gwei and ETH at $2,500, a single vote costing 44,180 gas would incur approximately $2.76 in transaction fees. In a national election with tens of millions of voters, public gas costs would be financially prohibitive. This empirical finding reinforces our design decision to deploy on a consortium Proof-of-Authority network where G_{price} is fixed to zero, eliminating all transaction fees for voters.

7.4 Security Vulnerability Analysis

The smart contract was evaluated against common smart contract vulnerabilities:

- **Reentrancy Attacks:** The contract executes state updates (`hasVoted = true`) before making external calls or emitting events. Furthermore, the contract does not transfer native Ether, completely neutralizing reentrancy vectors.
- **Integer Overflow / Underflow:** The contract uses Solidity 0.8.20, which automatically reverts on arithmetic overflows and underflows without requiring third-party libraries like `SafeMath`.
- **Denial-of-Service (DoS) via Unbounded Loops:** State variables and mappings are accessed via direct index lookup ($O(1)$ complexity). There are no unbounded `for` or `while` loops that could exceed block gas limits.
- **Front-Running and Miner Extractable Value (MEV):** In a private PoA network with pre-authorized institutional sealers and zero gas fees, priority fee manipulation and front-running are eliminated.

Chapter 8

Feasibility Study

This chapter assesses the technical, operational, and economic feasibility of deploying the proposed blockchain voting system, accompanied by the project implementation schedule.

8.1 Technical Feasibility

The technical feasibility of the proposed system is validated by the maturity of the underlying open-source technologies:

- **Solidity and the EVM:** The Ethereum Virtual Machine is the most battle-tested decentralized computing environment in existence, with comprehensive developer toolchains (Hardhat, Remix, Truffle).
- **Client Integration:** Web3.js and MetaMask provide standardized, cross-platform APIs supported by all modern browsers.
- **Hardware Requirements:** Node validator clients (e.g., Geth, Besu) operate efficiently on standard off-the-shelf workstation hardware, requiring no specialized mining rigs or graphics processing units (GPUs).

8.2 Operational Feasibility

From an operational perspective, the system simplifies election logistics:

- **Voter Usability:** Voters require only a web browser with the MetaMask extension. The intuitive interface provides clear, sequential guidance from wallet connection to ballot confirmation.
- **Administrative Simplicity:** Candidate nomination and voter whitelisting are streamlined through the dedicated administrative dashboard (`admin.html`).
- **Public Trust and Auditability:** Results are computed directly by the blockchain and displayed publicly in real time, eliminating allegations of counting room manipulation.

8.3 Economic Feasibility

Traditional national elections incur billions of dollars in expenditures for ballot printing, secure ballot transport, security personnel, and manual counting centers. In contrast:

- **Software Licensing:** Solidity, Geth, React.js, Web3.js, and MetaMask are 100% free, open-source software (FOSS), eliminating software licensing costs.
- **Hardware Infrastructure:** A consortium of 5–10 validator nodes run by independent institutions requires minimal cloud or on-premise server infrastructure.
- **Zero-Gas Economics:** By deploying on a private PoA network, ongoing transaction gas costs are zero.

8.4 Project Implementation Timeline

Figure 8.1 presents the project implementation Gantt chart spanning the academic year from July 2026 to March 2027.

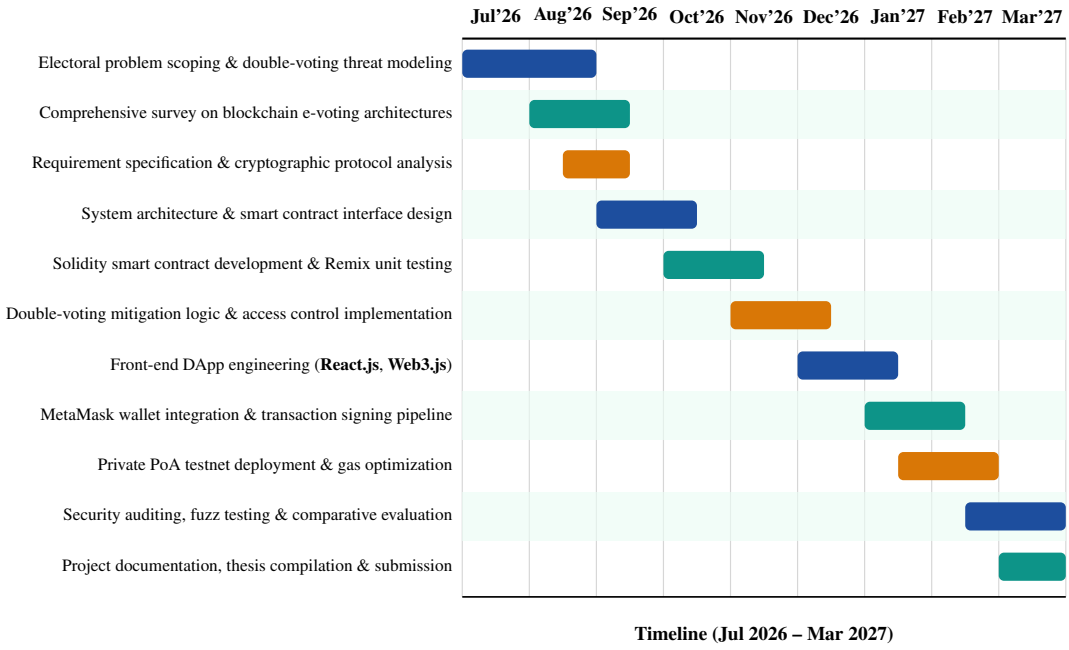


Figure 8.1: Gantt chart illustrating the project development timeline and milestone schedule (Jul 2026 – Mar 2027).

The project progressed through eleven structured milestones: problem scoping and threat modeling (Jul–Aug '26), literature survey (Aug–Sep '26), requirement analysis (Sep–Oct '26), system architecture design (Oct–Nov '26), Solidity contract development (Nov–Dec '26), double-voting mitigation implementation (Dec '26–Jan '27), React/Web3 front-end engineering (Jan–Feb '27), MetaMask integration (Jan–Feb '27), private PoA testnet deployment (Feb–Mar '27), security testing and comparative analysis (Feb–Mar '27), and final documentation and thesis compilation (Mar '27).

Chapter 9

Conclusion and Future Scope

This chapter concludes the report by summarizing key empirical and architectural achievements, addressing real-world limitations, and outlining prospective research trajectories.

9.1 Summary of Achievements

This project presented the design, implementation, and analysis of a **Block Chain Based Electronic Voting System**, realized as an Ethereum-based decentralized application (Voting DApp) with smart-contract-level double-voting mitigation. The primary contributions and achievements of this work include:

- **Guaranteed Double-Voting Mitigation:** Designed and implemented an atomic smart contract acceptance predicate (Accept(v)) in Solidity that makes duplicate voting computationally impossible. Once a ballot is cast, the voter's participation flag is atomically mutated within the same transaction, preventing race conditions or direct RPC replay attacks.

- **Role-Based Administrative Security:** Implemented strict cryptographic authorization boundaries separating the election administrator (solely empowered to add candidates and whitelist voter addresses) from voters (authorized solely to submit a single ballot).

- **Seamless Web3 Client Integration:** Developed an intuitive front end using React.js and Web3.js that interfaces with the MetaMask browser wallet, providing secure ECDSA digital signature generation without exposing private credentials.

- **Deterministic Zero-Cost Consortium Consensus:** Analyzed consensus models and established the superiority of a consortium Proof-of-Authority (PoA) architecture, providing sub-2-second block latency, high transaction throughput, and zero gas cost for voters.
- **Comprehensive Literature Survey:** Synthesized a rigorous comparative taxonomy of fifteen recent state-of-the-art blockchain voting architectures across biometrics, blind signatures, and smart contracts, identifying critical scalability and voter verification trade-offs.

9.2 System Limitations

Despite its significant security advantages over centralized electronic voting, several operational limitations remain:

1. **Private Key Custody and Digital Literacy:** The system assumes that voters are capable of securely managing their MetaMask wallet and seed phrases. Lost private keys result in disenfranchisement, while stolen private keys permit credential theft.
2. **Client Device Vulnerabilities:** While the blockchain ledger is tamper-proof, the voter's local computer or browser could be compromised by keyloggers, screen recording malware, or malicious browser extensions that spoof candidate selections before transaction signing.
3. **Public Ledger Pseudonymity vs Ballot Secrecy:** On a fully transparent Ethereum ledger, addresses are pseudonymous rather than completely anonymous. If an adversary links a physical citizen's identity to their public wallet address v, the citizen's vote selection c is permanently observable on-chain.
4. **Lack of Coercion Resistance:** The system allows voters to cast ballots from unmonitored remote environments. Without isolated physical voting booths, voters can be coerced or induced to vote under third-party observation.

9.3 Future Scope and Research Trajectories

To address these limitations, future research will explore the following advanced directions:

- **Zero-Knowledge Proofs (zk-SNARKs):** Integrate zk-SNARK protocols (such as Groth16 or PLONK) to enable voters to prove eligibility and cast a certified ballot without revealing their identity or candidate selection on the public ledger, achieving complete cryptographic ballot privacy.
- **Blind Signature Protocols:** Implement an ElGamal or RSA blind signature issuance phase during voter registration, cryptographically uncoupling voter whitelisting from ballot casting.
- **Multimodal Biometric Identity Verification:** Incorporate decentralized facial and fingerprint biometric hashing during registration (without storing raw biometric templates on-chain) to eliminate key-sharing and credential sale attacks.
- **Layer-2 Rollups and Sharding:** Deploy Layer-2 Zero-Knowledge Rollups (zk-Rollups) or Optimistic Rollups to batch thousands of ballot transactions off-chain, scaling throughput to tens of thousands of TPS suited for national elections.
- **Decentralized Off-Chain Storage with IPFS:** Integrate the InterPlanetary File System (IPFS) to store candidate manifestos, multimedia profiles, and audit receipts in a decentralized content-addressed storage layer.

References

[1] P. Asha, P. G. Reddy, R. A. Canessane, J. Refonaa, L. Padma Suresh, and R. S. Nancy Noella, "A Secure Approach to Mitigate Double Voting in Electronic Voting Systems using Blockchain Technology," in *Proc. 2025 8th International Conference on Circuit, Power and Computing Technologies (ICCPCT)*, Kollam, India, 2025, pp. 115–120, doi: 10.1109/ICCPCT65132.2025.11176586.

[2] M. S. Farooq, U. Iftikhar, and A. Khelifi, "A Framework to Make Voting System Transparent Using Blockchain Technology," *IEEE Access*, vol. 10, pp. 54102–54118, 2022, doi: 10.1109/ACCESS.2022.3175783.

[3] B. Shahzad and J. Crowcroft, "Trustworthy Electronic Voting Using Adjusted Blockchain Technology," *IEEE Access*, vol. 7, pp. 24477–24488, 2019, doi: 10.1109/ACCESS.2019.2895670.

[4] N. Kshetri and J. Voas, "Blockchain-Enabled E-Voting," *IEEE Software*, vol. 35, no. 4, pp. 95–99, Jul./Aug. 2018, doi: 10.1109/MS.2018.2801546.

[5] D. Seftyanto, A. Amiruddin, and A. R. Hakim, "Design of Blockchain-Based Electronic Election System Using Hyperledger: Case of Indonesia," in *Proc. 2019 11th International Conference on Information Technology and Electrical Engineering (ICITEE)*, Pattaya, Thailand, 2019, pp. 1–6, doi: 10.1109/ICITEE.2019.8929965.

[6] L. A. Ajao, B. U. Umar, H. O. Ohize, E. M. Dogo, E. Esenogho, and M. Cameron, "Blockchain Integration With Multimodal Biometric Authentication System for Secure Smart Verifiable Electronic Voting System," *IEEE Access*, vol. 13, pp. 14201–14220, 2025, doi: 10.1109/ACCESS.2025.3530012.

[7] W. A. Mahmood, J. Waleed, A. R. Abbas, H. Alaskar, M. Altulyan, and A. J. Hussain, "Intelligent Gesture-Enhanced Blockchain Voting: A New Era of Secure and Accessible E-Voting," *IEEE Access*, vol. 12, pp. 88310–88325, 2024,

doi: 10.1109/ACCESS.2024.3418892.

[8] J. C. Pérez García, A. Benslimane, and S. Boutalbi, "Blockchain-Based System for E-Voting Using Blind Signature Protocol," in *Proc. IEEE Global Communications Conference (GLOBECOM)*, Madrid, Spain, 2021, pp. 1–6, doi: 10.1109/GLOBECOM46510.2021.9685957.

[9] H. Baniata and G. Caluna, "BP-Vot: Blockchain-Based e-Voting Using Smart Contracts, Differential Privacy, and Self-Sovereign Identities," *IEEE Transactions on Emerging Topics in Computing*, vol. 13, no. 1, pp. 45–58, Jan.–Mar. 2025, doi: 10.1109/TETC.2024.3411200.

[10] A. Benabdallah, A. Audras, L. Coudert, N. El Madhoun, and M. Badra, "Analysis of Blockchain Solutions for E-Voting: A Systematic Literature Review," *IEEE Access*, vol. 10, pp. 70746–70759, 2022, doi: 10.1109/ACCESS.2022.3187688.

[11] A. Quzmar, M. Qatawneh, and S. Al-Maaitah, "E-Voting System Based on Blockchain Technology: A Survey," *International Journal of Computer Applications*, vol. 183, no. 42, pp. 1–10, Nov. 2021.

[12] A. Sah, A. Kumar, and B. Bhushan, "Leveraging Blockchain Technology for Secure Online Voting Systems: A Comprehensive Review," *Journal of Network and Computer Applications*, vol. 222, p. 103810, Feb. 2025, doi: 10.1016/j.jnca.2024.103810.

[13] R. Barelli, M. D'Onghia, and S. Longari, "Toward Secure Electronic Voting: A Survey on E-Voting Systems and Attacks," *ACM Computing Surveys*, vol. 57, no. 2, pp. 1–38, Feb. 2025, doi: 10.1145/3698780.

[14] C. C. Wermelinger, A. M. Mariano, and J. M. da Silva, "Trust and Security in Critical Digital Infrastructures: A Conditional Analysis of Acceptance and Intention to Change in Electronic Voting Systems," *Government Information Quarterly*, vol. 43, no. 1, p. 101890, Jan. 2026, doi: 10.1016/j.giq.2025.101890.

[15] S. Chaudhary, S. Shah, R. Kakkar, R. Gupta, A. Alabdulatif, S. Tanwar, G. Sharma, and P. N. Bokoro, "Blockchain-Based Secure Voting Mechanism Underlying 5G Network: A Smart Contract Approach," *IEEE Access*, vol. 11, pp. 60521–60535, 2023, doi: 10.1109/ACCESS.2023.3285149.